



Разработали студенты 3 курса, группы РПО
23/2, колледжа «IT TOP College»:

Соколов Иван
Мучаев Санчир

ШАШКИ ПО СЕТИ

ЦЕЛЬ ПРОЕКТА

Проект разработан для демонстрации полного жизненного цикла разработки ПО: от требований до тестирования и пользовательской документации

Требования



Проектирование



Код



Тестирование



Документирование

Основная цель - создать приложение, где два игрока могут играть в шашки через локальное TCP-соединение.

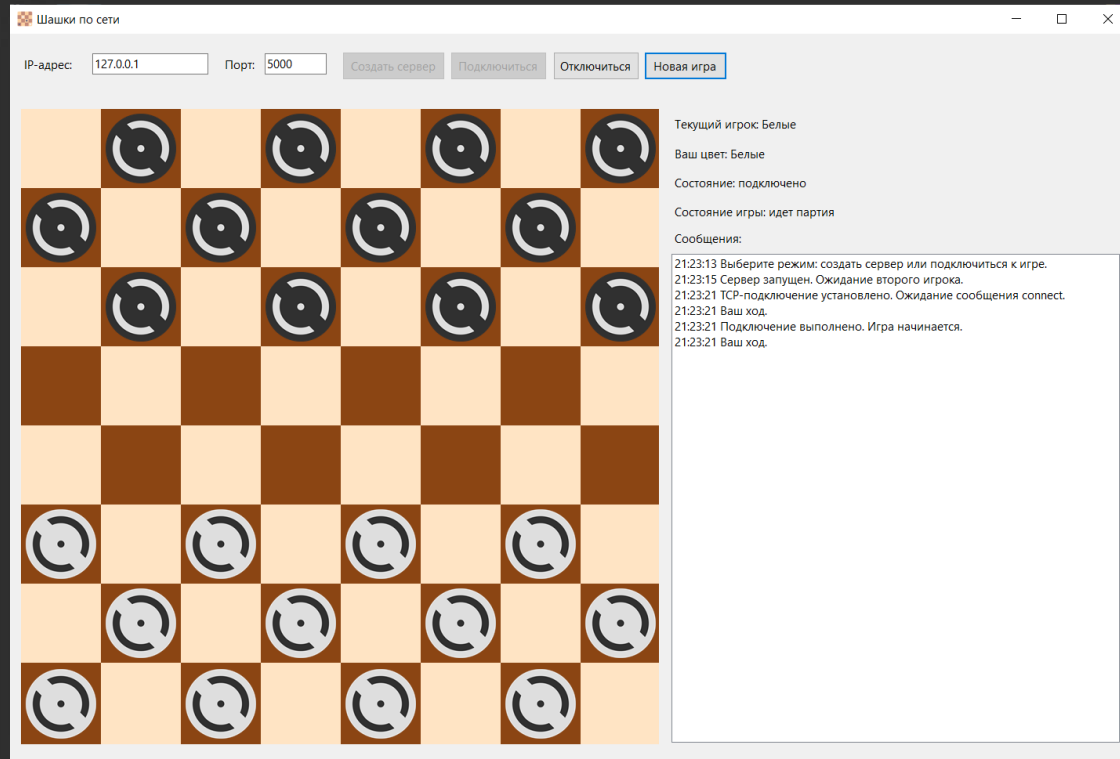
НАЗНАЧЕНИЕ

Приложение позволяет:

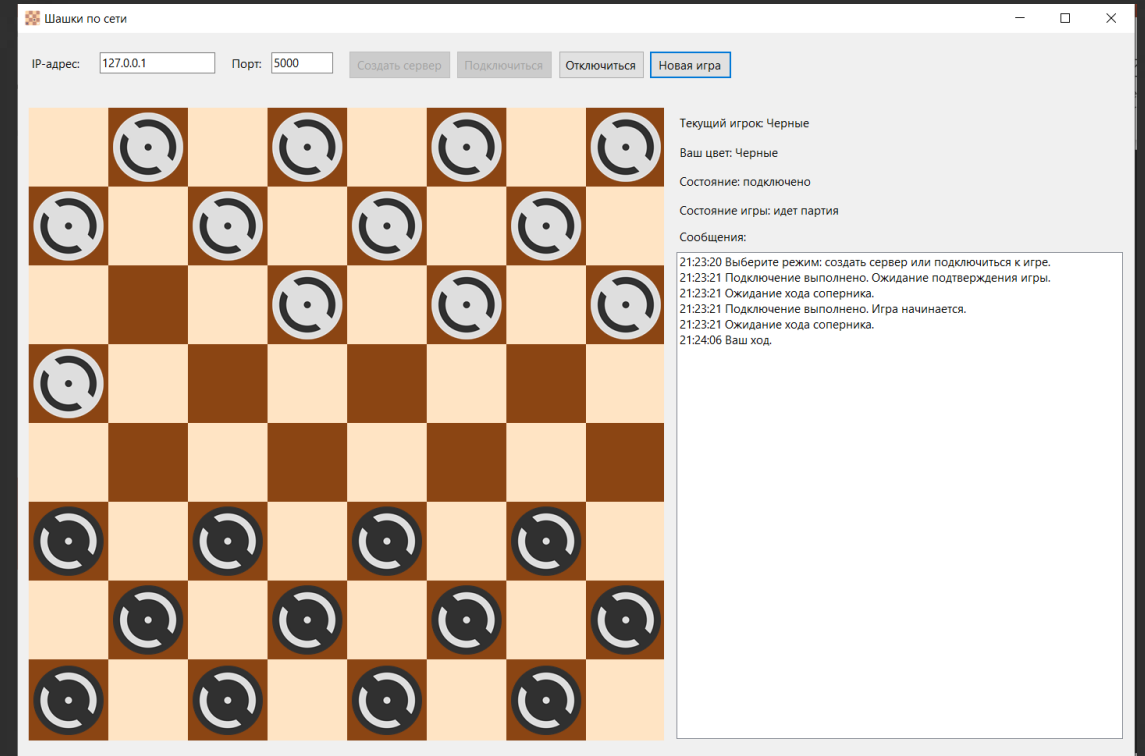
- создать TCP-сервер
- подключиться к серверу по IP и порту;
- играть в шашки по очереди
- проверять корректность ходов
- синхронизировать доску между двумя игроками
- начинать новую игру



МАТЧ 2 ИГРОКОВ:



Игрок 1



Игрок 2

СТЕК



Основной язык
разработки



Графический
интерфейс



Сетевое
взаимодействие



Установщик



Платформа
приложения



Тестирование



Контроль
версий



АРХИТЕКТУРА

UI/MainForm

интерфейс,
действия

GameLogic

правила игры и
состояние партии

Network

сервер, клиент,
сообщения

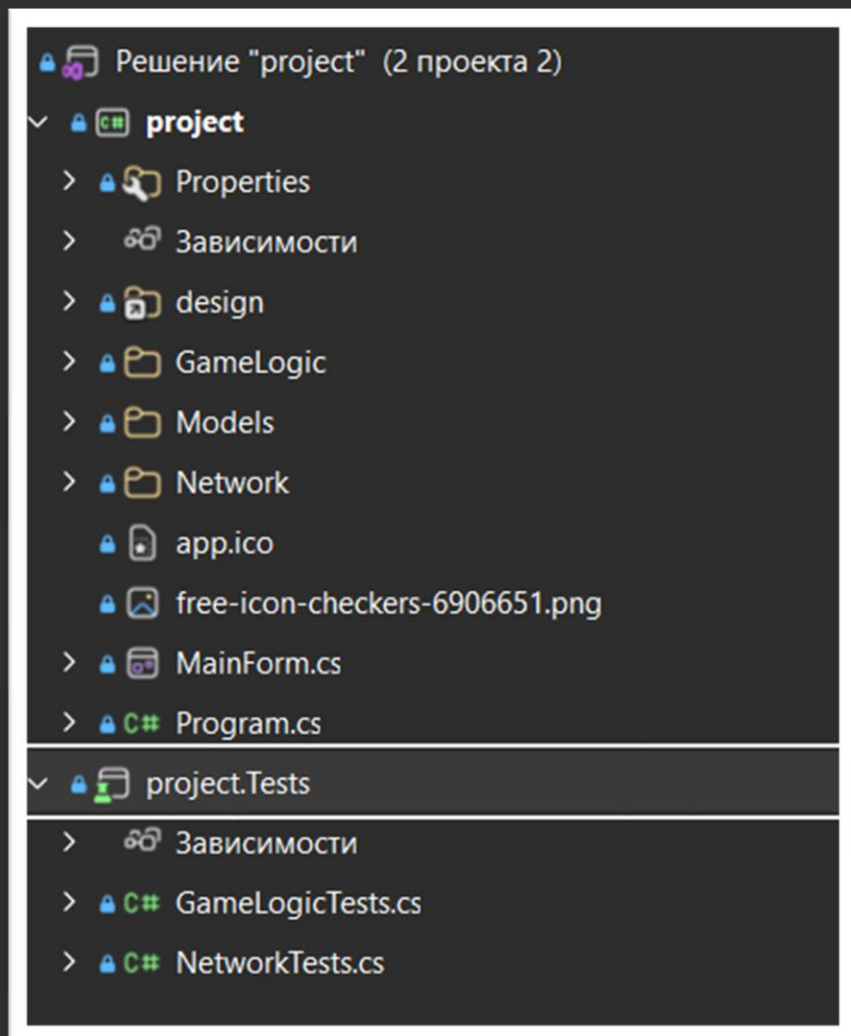
Models

модели данных

Tests

проверка логики
и сети

СТРУКТУРА



MainForm.cs — главное окно

GameLogic/ — игровая логика

Models/ — модели хода, шашки, состояния

Network/ — TCP-сервер и TCP-клиент

project.Tests/ — тесты

installer/ — сценарий установщика

publish/ — готовая публикация

КЛАССЫ

MainForm

связывает
интерфейс, игру и
сеть

CheckersGame

управляет партией
игроков

GameBoard

хранит состояние
доски

MoveValidator

проверяет
допустимость ходов

TcpGameServer

запускает сервер

TcpGameClient

подключается к
серверу

NetworkMessage

описывает сетевые
сообщения

ИГРОВАЯ ЛОГИКА

Игровая логика отвечает за:

- начальную расстановку
- выбор текущего игрока
- проверку ходов
- выполнение взятий
- превращение шашки в дамку
- цепные взятия
- определение победителя

Основные методы:

- StartNewGame()
- TryMakeMove(Move move)
- GetBoardSnapshot()
- CheckWinner()

Ссылка: 11

```
public MoveResult TryMakeMove(Move move)
{
    if (state != GameState.Playing)
    {
        return MoveResult.Failed("Игра сейчас недоступна.");
    }

    if (requiredCaptureFrom is not null)
    {
        if (move.FromRow != requiredCaptureFrom.Row || move.FromCol != requiredCaptureFrom.Col)
        {
            return MoveResult.Failed("Необходимо продолжить взятие той же шашкой.");
        }
    }

    if (!validator.IsValidMove(move, currentPlayer))
    {
        return MoveResult.Failed("Ход невозможен: нарушены правила игры.");
    }

    var result = ApplyMove(move);
    if (!result.Success)
    {
        return result;
    }

    var winner = CheckWinner();
    if (winner.HasValue)
    {
        result.Winner = winner;
        state = winner.Value == PlayerColor.White ? GameState.WhiteWon : GameState.BlackWon;
        GameStateChanged?.Invoke(state);
    }

    BoardChanged?.Invoke();
    return result;
}
```

ПРОВЕРКА ХОДА

Перед выполнением хода проверяется:

- находится ли клетка в пределах доски
- есть ли шашка в исходной клетке
- принадлежит ли шашка текущему игроку
- свободна ли целевая клетка
- выполняется ли ход по диагонали
- нет ли обязательного взятия

```
Ссылка: 1
public bool IsValidMove(Move move, PlayerColor player)
{
    if (!board.IsInsideBoard(move.FromRow, move.FromCol) || !board.IsInsideBoard(move.ToRow, move.ToCol))
    {
        return false;
    }

    var piece = board.GetPiece(move.FromRow, move.FromCol);
    if (piece is null)
    {
        return false;
    }

    if (!ValidatePieceOwner(move, player))
    {
        return false;
    }

    if (!board.IsCellEmpty(move.ToRow, move.ToCol))
    {
        return false;
    }

    if (!move.IsDiagonal())
    {
        return false;
    }

    return piece.IsKing() ? ValidateKingMove(move, piece.Color) : ValidateManMove(move, piece.Color);
}
```

СЕТЕВОЕ ВЗАИМОДЕЙСТВИЕ

Сеть работает по принципу:

- сервер запускает **TcpListener**
- клиент подключается через **TcpClient**
- сообщения передаются построчно в JSON
- каждый ход отправляется как сообщение move
- при отключении отправляется disconnect

Пример сообщения:

JSON



```
{ "type": "move", "fromRow": 5, "fromCol": 0, "toRow": 4, "toCol": 1 }
```

СОСТОЯНИЯ

WaitingForConnection — ожидание подключения

Playing — идет партия

WhiteWon — победили белые

BlackWon — победили черные

ConnectionError — ошибка соединения



ТЕСТИРОВАНИЕ

Для проверки проекта был создан отдельный тестовый проект project.Tests.
Тестирование выполнено с использованием фреймворка xUnit.

GameLogicTests — 10 тестов

- начальная расстановка шашек
- корректные и некорректные ходы
- обязательное и последовательное
взятие
- превращение шашки в дамку
- определение победителя

NetworkTests — 8 тестов

- сериализация сетевых сообщений в
JSON
- сообщения move, connect, newGame,
disconnect
- передача хода через локальное TCP-
соединение
- обработка потери соединения

ИТОГ

В результате был разработан учебный проект «Шашки по сети» — настольное приложение для игры двух пользователей через локальное TCP-соединение. Приложение позволяет создать сервер, подключиться к нему и провести партию с проверкой правил.

Что реализовано:

- игровое поле 8x8 и начальная расстановка
- проверка ходов, взятий, дамок и победителя
- сетевой обмен ходами через TCP
- обработка ошибок и потери соединения

Что подготовлено:

- проектная и техническая документация
- модульные и сетевые тесты
- публикация приложения
- установщик через Inno Setup

Проект показывает полный цикл разработки ПО: от требований и проектирования до реализации, тестирования, публикации и подготовки установщика.

**СПАСИБО ЗА
ВНИМАНИЕ**

